WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Finite State Machines

Lecture 10: sequential logic, Moore and Mealy machines, and simplification

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Sequential logic

02 Finite state machines

03 Simplifying state machines

04 Summary

01

Sequential logic

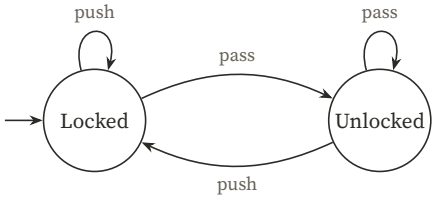
Beyond combinational logic

- A combinational circuit computes its outputs from its *present* inputs only (Lecture 07).
- A sequential circuit may also take the *past* into account. Combinational logic is the special case that ignores it.
- The sequential circuits a computer needs most: the **clock**, the **memory** and the **counter** (Unit 11–13).

> **Remark**
>
> Sequential circuits come in two kinds. *Synchronous* ones change state only at the ticks of a clock; *asynchronous* ones whenever their inputs change. The difference becomes clear with the clock and memory in the next lectures.

The turnstile

- Two actions: use a ticket (*pass*) and turn the arm (*push*).
- Will the arm turn if someone pushes it? The present action alone cannot say: it depends on whether a ticket was used before.
- The turnstile must remember one fact about the past: it is locked or unlocked.



Definition (State)

A *state* is a stored summary of the past: everything that must be remembered to decide what happens next.

The mystic box

A box with two buttons, *On* and *Off*. A pressed button sends 1, a released one 0. We never press both. The box should turn green on *On*, red on *Off*, and keep its colour otherwise.

Time	0	1	2	3	4	5	6	7	8	9	10	11
Input (On, Off)	10	00	00	00	01	00	00	10	00	01	01	00
Colour	G	G	G	G	R	R	R	G	G	R	R	R

- Input 00 appears at times 1, 2, 3, 5, 6, 8 and 11, with colour G at 1, 2, 3, 8 and R at 5, 6, 11.
- A combinational circuit gives one output per input. Make 00 green, and times 5, 6, 11 are wrong; make it red, and times 1, 2, 3, 8 are wrong.

The box needs a state: the colour it showed last.

02

Finite state machines

Finite state machines

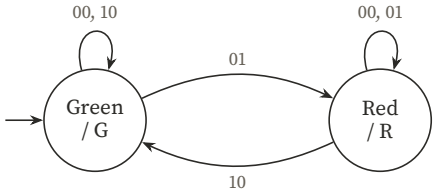
Definition (Finite state machine)

A *finite state machine* (FSM) has a finite set of states, one of them the *initial* state, inputs and outputs, and two rules:

- the *transition* rule: next state = $f(\text{state}, \text{input})$;
- the *output* rule, in one of two kinds:
  - **Moore**: output = $g(\text{state})$;
  - **Mealy**: output = $g(\text{state}, \text{input})$.

The rule $s_{t+1} = f(s_t, x_t)$ from Lecture 01 was the transition rule of an FSM. The push-button lamp there was one, with two states.

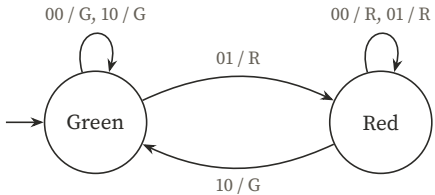
The mystic box as a Moore machine



- Each state shows its name and its output: *state / output*.
- Arrows are labelled with the input (On, Off) that takes them.
- The arrow from nowhere marks the initial state.

State	next on 00	next on 01	next on 10	output
Green	Green	Red	Green	G
Red	Red	Red	Green	R

The mystic box as a Mealy machine



- The outputs move from the states to the arrows: *input / output*.
- In Red, input 10 outputs G while it moves to Green.

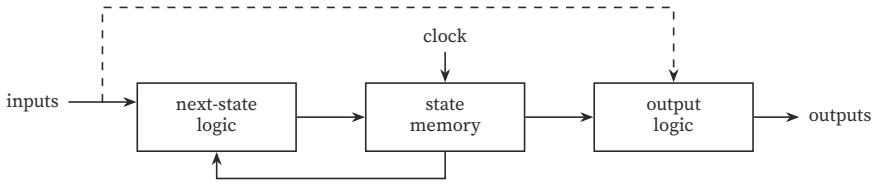
Moore

- Outputs change only when the state changes.
- No path from an input straight to an output.

Mealy

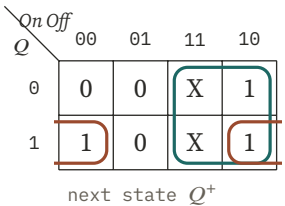
- Outputs react to the input at once.
- Often needs fewer states.

From a state machine to a circuit



- The state is stored in memory, as bits `(Unit 11–12)`. The next-state and output logic are combinational.
- The dashed input to the output logic is there only in a Mealy machine.
- States become bits: with 2 states, 1 bit is enough. A one-hot code uses one bit per state, which makes the logic simpler (Lecture 08).

The mystic box as a circuit



- Encode Green as $Q = 1$, Red as $Q = 0$. The output is just Q.
- Next state: 1 after *On*, 0 after *Off*, otherwise Q. Both buttons at once never happens: X.

$$Q^+ = On + Off' \, Q$$

Coming back: a one-bit memory with a ‘set’ and a ‘reset’ input. This is the SR latch of the next lecture (Unit 11).

03

Simplifying state machines

Equivalent states

- Two states are equivalent if no sequence of inputs can tell them apart: from either one, every input sequence gives the same outputs.
- Equivalent states can be merged into one. Fewer states need fewer bits of memory and simpler logic.
- The book works on the state table, and merges states whose rows are identical.

Simplifying a Moore machine

1. Find two states p and q with the same output and the same next state for every input.
2. They are equivalent: delete q, and replace q by p everywhere.
3. Repeat until no such pair is left.

Example: a Moore machine

state	0	1	out
a	b	c	1
b	b	d	0
c	b	c	1
d	b	c	0
e	b	a	1

the book's Table 5.3

state	0	1	out
a	b	a	1
b	b	d	0
d	b	a	0
e	b	a	1

c merged into a

state	0	1	out
a	b	a	1
b	b	d	0
d	b	a	0

e merged into a

- a and c: output 1, next states b and c. Merge c into a.
- Now a, d and e all go to b and a, but only a and e output 1. Merge e into a.
- b and d have the same output, but on input 1 they go to d and a: not mergeable. Five states became three.

Simplifying a Mealy machine

In a Mealy machine the outputs are on the transitions, so compare (input, next state, output) for every input.

state	0	1
a	b / 1	c / 1
b	b / 0	d / 1
c	b / 1	c / 1
d	e / 0	c / 1
e	e / 0	a / 1

the book's Table 5.6

state	0	1
a	b / 1	a / 1
b	b / 0	d / 1
d	e / 0	a / 1
e	e / 0	a / 1

c merged into a

state	0	1
a	b / 1	a / 1
b	b / 0	d / 1
d	d / 0	a / 1

e merged into d

The simplified machine gives the same outputs as the original for every input sequence.

When identical rows are not enough

state	0	1	out
p	r	q	0
q	r	p	0
r	r	p	1

- The rows of p and q differ, yet the two states are equivalent: they only send each other back and forth.
- The full method, *partition refinement*: group the states by output, then split any group whose states lead to different groups on some input. Repeat until nothing splits.

Here: $\{p, q\}$ and $\{r\}$ by output, and nothing splits, so p and q merge. The identical-rows rule finds a subset of these merges.

Try it

1. Draw a Moore machine whose output is 1 when its last two inputs were both 1.
   Three states: no 1 yet (out 0), one 1 (out 0), two or more 1s (out 1). Every 0 goes back to the first.
2. Now draw it as a Mealy machine. How many states does it need?
   Two: ‘last input 0’ and ‘last input 1’. In the second, input 1 outputs 1.
3. Simplify this Moore machine: p: $0 \to q$, $1 \to r$, out 0; q: $0 \to p$, $1 \to s$, out 1; r: $0 \to q$, $1 \to r$, out 0; s: $0 \to p$, $1 \to s$, out 1.
   $p \equiv r$ and $q \equiv s$: two states are enough.

04

Summary

Summary

- Sequential circuits use the past as well as the present. The past is kept as a state.
- A finite state machine: finitely many states, an initial state, a transition rule and an output rule.
- Moore machines take the output from the state; Mealy machines from the state and the input.
- As a circuit: state memory, next-state logic and output logic.
- Equivalent states can be merged. Identical rows are a quick test; partition refinement finds every merge.

Before the next lecture

- Read the beginning of Chapter 5 and Section 5.1 of the book.
- Draw the turnstile as a Mealy machine whose output says whether the arm turns.
- Simplify the book's Moore machine in Table 5.3 with partition refinement, and check that you get the same three states.

> **Next lecture** (Unit 11)
>
> The clock, and memory: the SR latch and the D latch.